

**A SECURE HEALTH CARE DATA STORING WITH
BLOCKCHAIN TECHNIQUE IN DATABASE**

ABSTRACT

Storing and sharing health records through electronic systems pose security risks. To address them, several countries' regulations have established that healthcare information systems must fulfill security properties (confidentiality, access control, integrity, revocation and anonymity) and complementary ones. Upon tackling these issues, several proposals present security limitations and/or address specific properties only. We propose Sec-Health, a block chain-based protocol that secures health records, addressing all of the main security and complementary properties defined in current regulations. We show that Sec-Health is a suitable solution by analyzing it under several attack scenarios and describing how it overcomes the problems of existing solutions.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT LIST OF FIGURES LIST OF SYMBOLS LIST OF ABBREVIATIONS LIST OF TABLES	i v vii xi xii
1.	CHAPTER 1 : INTRODUCTION 1.1 GENERAL 1.2 OBJECTIVE 1.3 EXISTING SYSTEM 1.3.1EXISTINGSYSTEM DISADVANTAGES 1.3.2 LITERATURE SURVEY 1.4 PROPOSED SYSTEM 1.4.1 PROPOSED SYSTEM ADVANTAGES	
2.	CHAPTER 2 :PROJECT DESCRIPTION 2.1 GENERAL 2.2 METHODOLOGIES 2.2.1 MODULES NAME 2.2.2 MODULES EXPLANATION 2.2.3 MODULE DIAGRAM 2.2.4GIVEN INPUTAND EXPECTED OUTPUT 2.3 TECHNIQUE OR ALGORITHM	

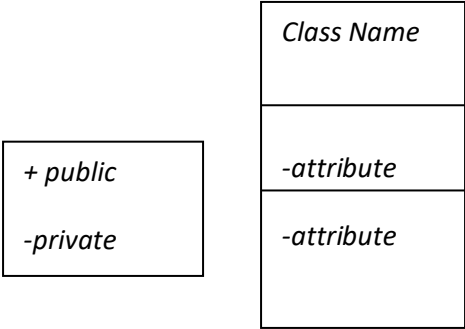
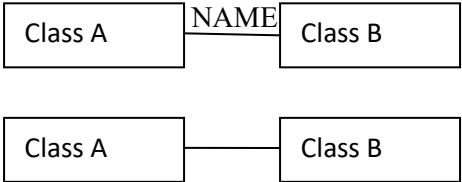
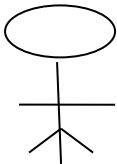
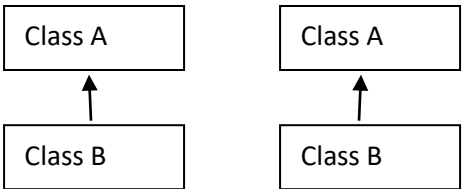
3.	CHAPTER 3 : REQUIREMENTS 3.1 GENERAL 3.2 HARDWARE REQUIREMENTS 3.3 SOFTWARE REQUIREMENTS	
4.	CHAPTER 4 : SYSTEM DESIGN 4.1 GENERAL 4.1.1 ACTIVITY DIAGRAM 4.1.2 USE CASE DIAGRAM 4.1.3 DATA FLOW DIAGRAM 4.1.4SEQUENCE DIAGRAM 4.1.5 COLLABORATION DIAGRAM 4.1.6CLASS DIAGRAM 4.1.7 SYSTEM ARCHITECTURE 4.1.8 OBJECT DIAGRAM 4.1.9 STATE DIAGRAM 4.1.10 COMPONENT DIAGRAM 4.1.11 E-R DIAGRAM 4.2 DATABASE DESIGN (ALL LEVEL	
5.	CHAPTER 5 :SOFTWARE SPECIFICATION 5.1 GENERAL FRONT END 5.2 FEATURES OF JAVA 5.2.1 THE JAVA FRAMEWORK 5.2.2 OBJECTIVE OF JAVA 5.2.3 COMPONETS OF JAVA FRMEWORK 5.3 FEATURES OF THE COMMON LANGUAGE RUNTIME	

	5.4 J2EE 5.4.1 EVALUTION OF J2EE 5.4.2 THE J2EE DATA ARCHITECTURE BACK END 5.6 FEATURES OF SQL SERVER	
6.	CHAPTER 6 : IMPLEMENTATION 6.1 GENERAL 6.2 IMPLEMENTATION 6.3 DATA BASE TABLE STRUCTURE	
7.	CHAPTER 7 : SNAPSHOTS 7.1 GENERAL 7.2 VARIOUS SNAPSHOTS	
8.	CHAPTER 8 : SOFTWARE TESTING 8.1 GENERAL 8.2 DEVELOPING METHODOLOGIES 8.3 TYPES OF TESTING	
9.	CHAPTER 9 : APPLICATIONS AND FUTURE ENHANCEMENT 9.1 GENERAL 9.2 APPLICATIONS 9.3 FUTURE ENHANCEMENTS	
10	CHAPTER 10 : 10.1 CONCLUSION 10.2 REFERENCES	

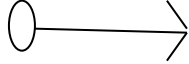
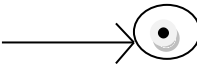
LIST OF FIGURES

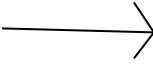
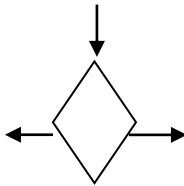
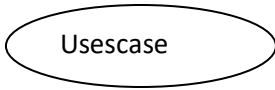
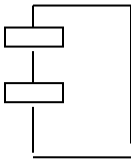
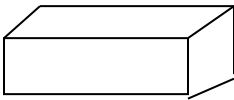
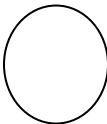
FIGURE NO	NAME OF THE FIGURE	PAGE NO.
2.3.2	Module Diagram	
4.2	Activity Diagram	
4.3	Use case Diagram	
4.4	Data flow diagram	
4.5	Sequence diagram	
4.6	Collaboration diagram	
4.7	Class diagram	
4.8	Architecture Diagram	
4.9	State Diagram	
4.1	Component Diagram	
4.12	E-R Diagram	

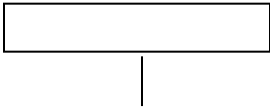
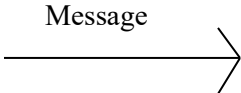
LIST OF SYMBOLS

S.NO	NOTATION NAME	NOTATION	DESCRIPTION
1.	Class		Represents a collection of similar entities grouped together.
2.	Association		Associations represents static relationships between classes. Roles represents the way the two classes see each other.
3.	Actor		It aggregates several classes into a single classes.
5.	Aggregation		Interaction between the system and external environment

--	--	--	--

5.	Relation (uses)	uses	Used for additional process communication.
6.	Relation (extends)		Extends relationship is used when one use case is similar to another use case but does a bit more.
7.	Communication		Communication between various use cases.
8.	State		State of the process.
9.	Initial State		Initial state of the object
10.	Final state		Final state of the object

11.	Control flow		Represents various control flow between the states.
12.	Decision box		Represents decision making process from a constraint
13.	Usecase		Interact ion between the system and external environment.
14.	Component		Represents physical modules which is a collection of components.
15.	Node		Represents physical modules which are a collection of components.
16.	Data Process/State		A circle in DFD represents a state or process which has been triggered due to some event or acion.
17.	External entity		Represents external entities

			such as keyboard,sensors,etc.
18.	Transition		Represents communication that occurs between processes.
19.	Object Lifeline		Represents the vertical dimensions that the object communications.
20.	Message		Represents the message exchanged.

LIST OF ABBREVIATION

S.NO	ABBREVIATION	EXPANSION
1.	DB	Database
2.	JVM	Java Virtual Machine
3.	JSP	Java Server Page
4.	CB	Collective Behavior
5.	SD	Social Dimension
6.	JRE	Java Runtime Environment
7.	SSD	Sparse Social Dimension
8.	LGP	Line Graph Partition

CHAPTER 1

INTRODUCTION

INTRODUCTION

Information technologies introduce a number of resources and benefits to the healthcare field. Electronic Health Records (EHRs), such as patient's medical history, are one of the most widely employed resources [1], providing a wide view of a patient's medical status. EHRs are commonly originated and shared with collaborators (e.g., physicians, nurses) through cloud computing systems, which results in a more convenient approach to managing such records. Cloud-based systems, however, introduce security challenges in healthcare [2]. A recent report shows that healthcare data breaches are highly common [3], wherein several of them are classed as unauthorized access, which may lead to inappropriate use of health records (e.g., unwanted advertisements or lower chances of conquering a job opportunity). Due to security vulnerabilities, various countries (e.g., USA, Brazil, and those from European Union) have established regulations defining health records as sensitive data that should be shared only under patient consent [4]. Such regulations define several requirements, which we call health record properties. For instance, only authorized collaborators should access health records (confidentiality and access control properties). Records must also be protected from unauthorized modifications (integrity property). Mechanisms must also exist to legitimately grant access to records in emergency situations (emergency access property), and to anonymized records for research purposes (anonymity property). Besides, the properties of access revocation and interoperability must also be addressed. Those properties motivate the design of solutions to secure healthcare information systems. A number of literature proposals provide schemes based on centralized servers to store and share health records (e.g., [5], [6]). The security of such solutions rely on the fact that the server is trusted not to disclose sensitive data, such as information related to user credentials and patient records. This results in a single point that, when compromised, can make the entire system fail. Moreover, these solutions address only a subset of health record properties, while not fulfilling fundamental ones. Other several works propose the use of decentralized approaches to secure health records (e.g., [7], [8]). For instance, blockchain [9], a technology that improves data security by allowing online data transactions in a decentralized fashion, has been adopted in different protocols. Although these schemes do not present a single point of failure, they still lack an integrated approach that covers all of the

aforementioned health records properties, then presenting security limitations. Therefore, there is a lack of proposals in the literature that address all of the properties and afford satisfactory security to health records. Motivated by the properties defined in regulations and the literature limitations, we propose Sec-Health, a protocol that secures health records by addressing all of their properties. In essence, Sec-Health is composed of a set of schemes, based on decentralized approaches (e.g., block chain and Interplanetary File System [10]) and cryptographic primitives (e.g., Cipher text-Policy Attribute-based Encryption [11] and public key encryption), which allow records to be stored and shared securely. Sec-Health fills the gap of the literature that lacks integrated approaches which fulfill all health records properties. It overcomes the security problems of proposals based on centralized servers and presents advantages over other decentralized solutions by covering not only the most addressed properties of health records (confidentiality, access control, and integrity), but also more challenging ones (e.g., emergency access, access revocation, and anonymity).

1.2 OBJECTIVE:

In this paper, we provide the following novel contributions a blockchain-based protocol (Sec-Health), based on our previous work, which enhances the schemes employed in the previous protocol to fulfill the security properties of confidentiality, access control, and integrity.

1.3 SCOPE OF PROJECT

The scope of project is To provide a background to clarify Sec-Health, we now introduce cryptographic primitives and present key concepts of block chain. Each stored file is addressed by its hash value, thus each file is unique in the system and the process of searching for files becomes more efficient. Retrieving a file requires performing a request using its hash.

1.4 EXISTING SYSTEM:

- The attacker may attempt to read a health record without authorization, modify a record while proving the altered version is legitimate, or completely delete a record without authorization.
- Also, an attacker may try to access a health record after its access rights for such record have been revoked. Finally, an attacker may attempt to break the anonymity of a record provided for research purposes.
- We consider that a collaborator who legitimately gains access to a health record does not disclose information on it to other parties

DRAWBACKS:-

- ✚ No Security authentication.
- ✚ Sensitive patient/ user data has a leaks a third party.
- ✚ the cloud cannot have a security of patient data.

1.5 LITERATURE SURVEY:

TITLE : Security framework for cloud based electronic health record (EHR) system.

AUTHOR : R. Ganiga, R. Pai, M. Pai, and R. Sinha

YEAR :2022

DESCRIPTION :

Health records are an integral aspect of any Hospital Management System. With newer innovations in technology, there has been a shift in the way of recording health information. Medical records which used to be managed using various paper charts have now become easier to organize and maintain, thereby increasing the efficiency of medical staff. The Electronic Health Records (EHR) System is becoming a high-tech medical management technology developed for the economic or emerging economic countries like India. In a national health system, the EHR integrates the Electronic Medical Records (EMR) in all collaborating hospitals through different networks. EHR gives healthcare professionals a way to share and manage patient data quickly and effectively. Due to the mass storage of confidential patient data, healthcare organizations are considered as one of the most targeted sectors by intruders. This paper proposes a security framework for EHR system, which takes into consideration the integrity, availability, and confidentiality of health records. The threats posed to the EHR system are modeled by STRIDE modeling tool, and the amount of risk is calculated using DREAD. The paper also suggests the security mechanism and countermeasures based on security standards, which can be utilized in an EHR environment. The paper shows that the utilization of the proposed methods effectively addresses security concerns such as breach of sensitive medical information.

TITLE : A robust and lightweight secure access scheme for cloud based Ehealthcare services

AUTHOR : M. Masud, G. S. Gaba, K. Choudhary, R. Alroobaea, and M. S. Hossain

YEAR : 2021

DESCRIPTION :

Traditional healthcare services have transitioned into modern healthcare services where doctors remotely diagnose the patients. Cloud computing plays a significant role in this change by providing easy access to patients' medical records to all stakeholders, such as doctors, nurses, patients, life insurance agents, etc. Cloud services are scalable, cost-effective, and offer a broad range of mobile access to patients' electronic health record (EHR). Despite the cloud's enormous benefits like real-time data access, patients' EHR security and privacy are major concerns. Since the information about patients' health is highly sensitive and crucial, sharing it over the unsecured wireless medium brings many security challenges such as eavesdropping, modifications, etc. Considering the security needs of remote healthcare, this paper proposes a robust and lightweight, secure access scheme for cloud-based E-healthcare services. The proposed scheme addresses the potential threats to E-healthcare by providing a secure interface to stakeholders and prohibiting unauthorized users from accessing information stored in the cloud. The scheme makes use of multiple keys formed through the key derivation function (KDF) to ensure end-to-end ciphering of information for preventing misuse. The rights to access the cloud services are provided based on the identity and the association between stakeholders, thus ensuring privacy. Due to its simplicity and robustness, the proposed scheme is the best fit for protecting data security and privacy in cloud-based E-healthcare services

TITLE : ‘Secure health data sharing for medical cyber-physical systems for the healthcare 4.0

AUTHOR : H. Qiu, M. Qiu, M. Liu, and G. Memmi

YEAR : 2022

DESCRIPTION :

The recent spades of cyber attacks have compromised end-users' data security and privacy in Medical Cyber-Physical Systems (MCPS) in the era of Health 4.0. Traditional standard encryption algorithms for data protection are designed based on a viewpoint of system architecture rather than a viewpoint of end-users. As such encryption algorithms are transferring the protection on the data to the protection on the keys, data safety, and privacy will be compromised once the key is exposed. In this paper, we propose a secure data storage and sharing method consisted of a selective encryption algorithm combined with fragmentation and dispersion to protect the data safety and privacy even when both transmission media (e.g. cloud servers) and keys are compromised. This method is based on a user-centric design that protects the data on a trusted device such as the end-users' smartphone and lets the end-user control the access for data sharing. We also evaluate the performance of the algorithm on a smartphone platform to prove efficiency

TITLE : 'E-health cloud security using timing enabled proxy re-encryption

AUTHOR : V. Vijayakumar, M. K. Priyan, G. Ushadevi, R. Varatharajan, G. Manogaran, and P. V. Tarare,

YEAR : 2022

DESCRIPTION :

The advancements made in cloud applications attracts the healthcare society. The analysis over Electronic Health Record (EHR) gains much importance among the research communities. As EHR contains sensitive information, the need of security and privacy is to be analyzed effectively. The services like outsourcing and increased computation has enhanced the usage of the digital technologies. The digital data interconnects with different network devices which lead to the study of reliability, scalability and security. Security is the major part of the outsourced databases. Prior works like searchable encryption and proxy re-encryption have been introduced by research communities. Still, the security requirements of the healthcare application are not yet achieved. Notwithstanding, prior searching encryption degrades in terms of storage computation. In this paper, we have proposed timing enabled proxy re-encryption system that permits users to access the data under certain time period T . Each identified user is defined with set of attributes and valid time period T . Security analysis in terms of decryption key compromised, Expiration of identity, conjunctive based similarity keyword search and lessened complexity of key update phase have been studied.

TITLE : 'BSF-EHR: Blockchain security framework for electronic health records of patients

AUTHOR : I. Abunadi and R. Kuma

YEAR : 2021

DESCRIPTION :

In the current epoch of smart homes and cities, personal data such as patients' names, diseases and addresses are often violated. This is frequently associated with the safety of the electronic health records (EHRs) of patients. EHRs have numerous benefits worldwide, but at present, EHR information is subject to considerable security and privacy issues. This paper proposes a way to provide a secure solution to these issues. Previous sophisticated techniques dealing with the protection of EHRs usually make data inaccessible to patients. These techniques struggle to balance data confidentiality, patient demand and constant interaction with provider data. Blockchain technology solves the above problems since it distributes information in a transactional and decentralized manner. The usage of blockchain technology could help the health sector to balance the accessibility and privacy of EHRs. This paper proposes a blockchain security framework (BSF) to effectively and securely store and keep EHRs. It presents a safe and proficient means of acquiring medical information for doctors, patients and insurance agents while protecting the patient's data. This work aims to examine how our proposed framework meets the security needs of doctors, patients and third parties and how the structure addresses safety and confidentiality concerns in the healthcare sector. Simulation outcomes show that this framework efficiently protects EHR data..

TITLE : A patient-centric health information exchange framework using blockchain technology

AUTHOR : Y. Zhuang, L. R. Sheets, Y.-W. Chen, Z.-Y. Shae, J. J. P. Tsai, and C.-R. Shyu,

YEAR : 2022

DESCRIPTION :

Health Information Exchange (HIE) exhibits remarkable benefits for patient care such as improving healthcare quality and expediting coordinated care. The Office of the National Coordinator (ONC) for Health Information Technology is seeking patient-centric HIE designs that shift data ownership from providers to patients. There are multiple barriers to patient-centric HIE in the current system, such as security and privacy concerns, data inconsistency, timely access to the right records across multiple healthcare facilities. After investigating the current workflow of HIE, this paper provides a feasible solution to these challenges by utilizing the unique features of blockchain, a distributed ledger technology which is considered “unhackable”. Utilizing the smart contract feature, which is a programmable self-executing protocol running on a blockchain, we developed a blockchain model to protect data security and patients’ privacy, ensure data provenance, and provide patients full control of their health records. By personalizing data segmentation and an “allowed list” for clinicians to access their data, this design achieves patient-centric HIE. We conducted a large-scale simulation of this patient-centric HIE process and quantitatively evaluated the model’s feasibility, stability, security, and robustness

1.6 PROPOSED SYSTEM:

- Sec-Health's goal is to address the aforementioned malicious behaviors by detecting, preventing and/or mitigating them.
- We propose Sec-Health, a block chain-based protocol that secures health records, addressing all of the main security and complementary properties defined in current regulations.
- We show that Sec-Health is a suitable solution by analyzing it under several attack scenarios and describing how it overcomes the problems of existing solutions

1.6.1 ADVANTAGES IN PROPOSED SYSTEM:

- ✚ Stronger authentication
- ✚ Every user (Patient) must register with the system.
- ✚ The access control module will verify the user's data from the security

CHAPTER 2

2.1 GENERAL

The registration of system users (patients, collaborators, call center attendants and research entities) follows common steps. Each of them generates an identifier and an encryption key pair to herself. In addition, the authority issues a CP-ABE secret key to the users that will access health records (patients, collaborators and research entities). Emergency servers follow a slightly different registration process in which they cooperatively generate a key pair from a threshold cryptosystem, wherein each server owns a share of the private key and an identifier. The identifiers and public keys of all participants are attested by the authority through the use of digital signatures and stored in the block chain so that any interested party in the system may access them.

2.2 METHODOLOGIES

Following modules involves

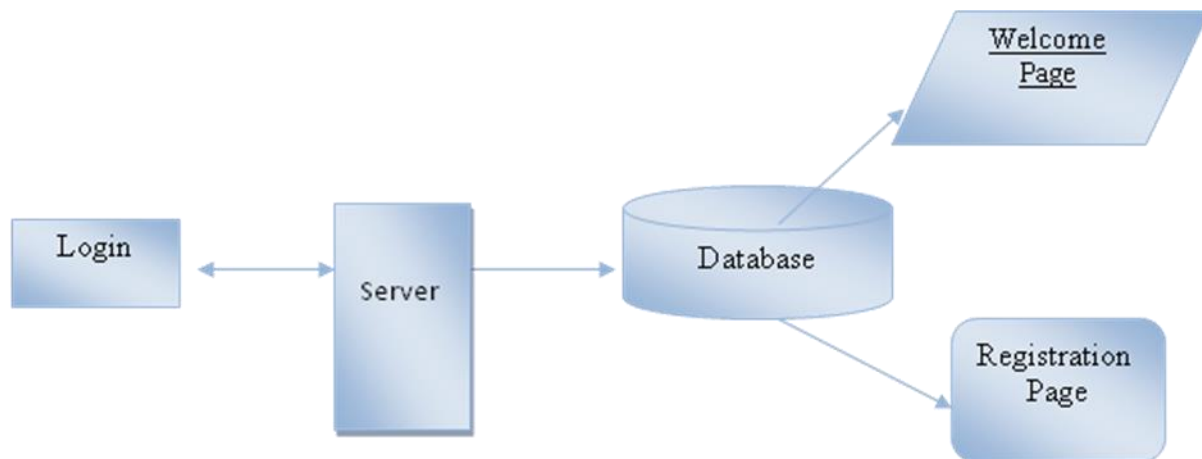
2.2.1 MODULES

There are Five modules they are

- 1.Hospital
- 2.Doctors
- 3.Patient
- 4.Emergency Doctor Admin
- 5.Emergency Patient login

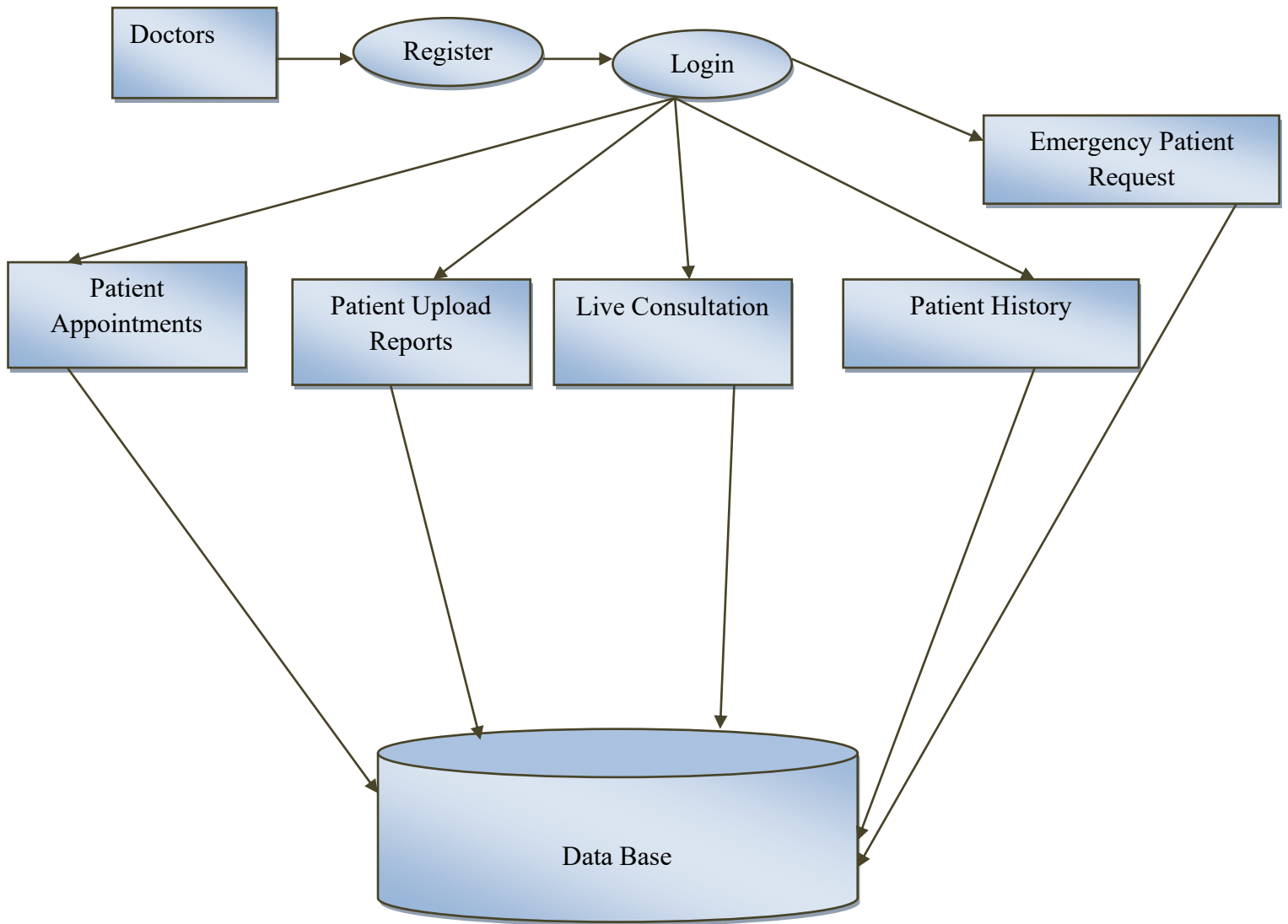
1. User interface

In this module we design the windows for the project. These windows are used for secure login for all users. To connect with server user must give their username and password then only they can able to connect the server. If the user already exists directly can login into the server else user must register their details such as username, password and Email id, into the server. Server will create the account for the entire user to maintain upload and download rate. Name will be set as user id. Logging in is usually used to enter a specific page.



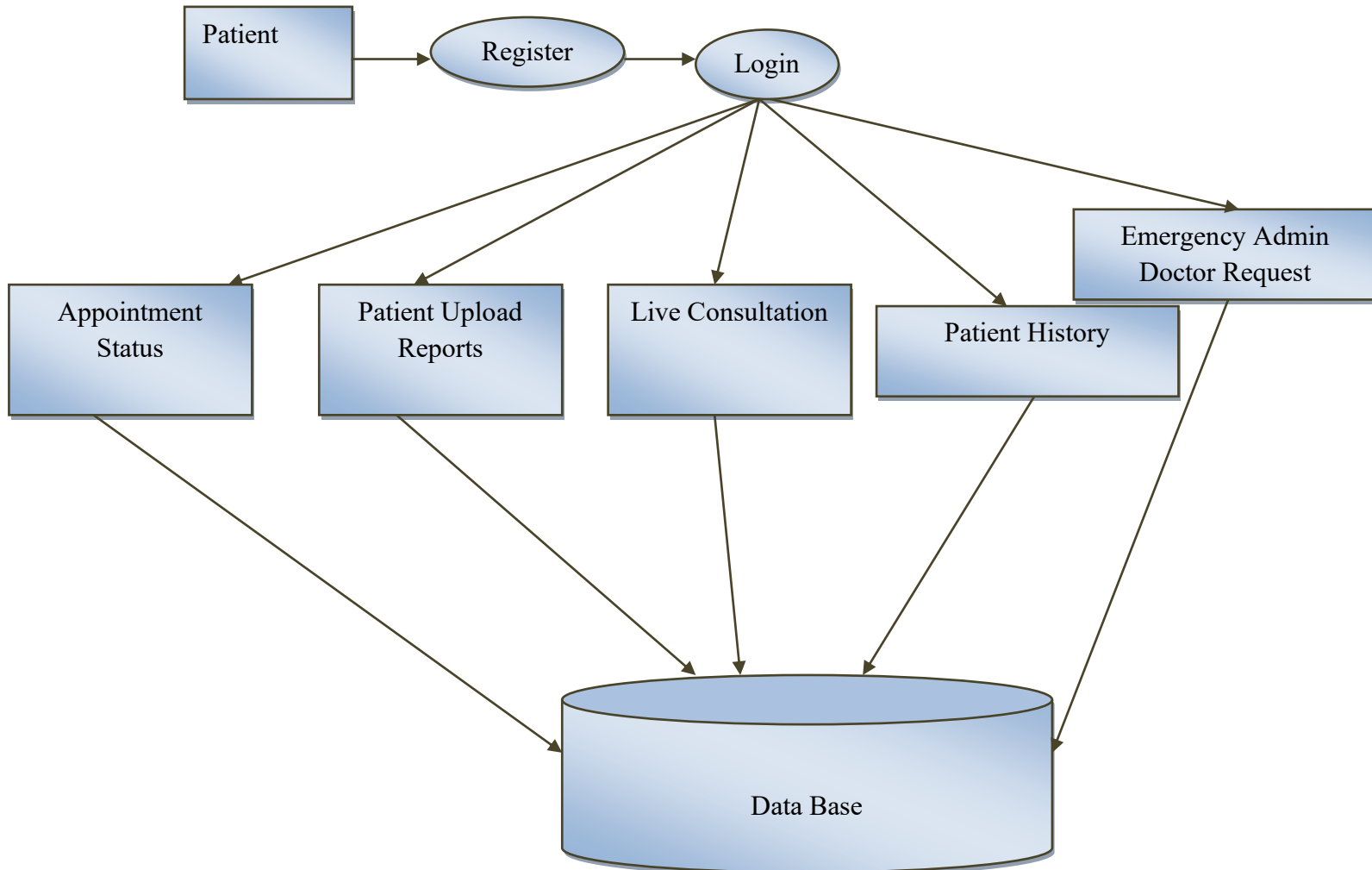
2.DOCTORS

There are doctor has a register with a details. Doctor has a takes permission from the hospital. Patients also a takes a permission from a doctor. Doctors have a patient appointment. Doctor has a view uploaded a report. Doctor has a live consultations between a doctor and patient. Doctor have a patient history.



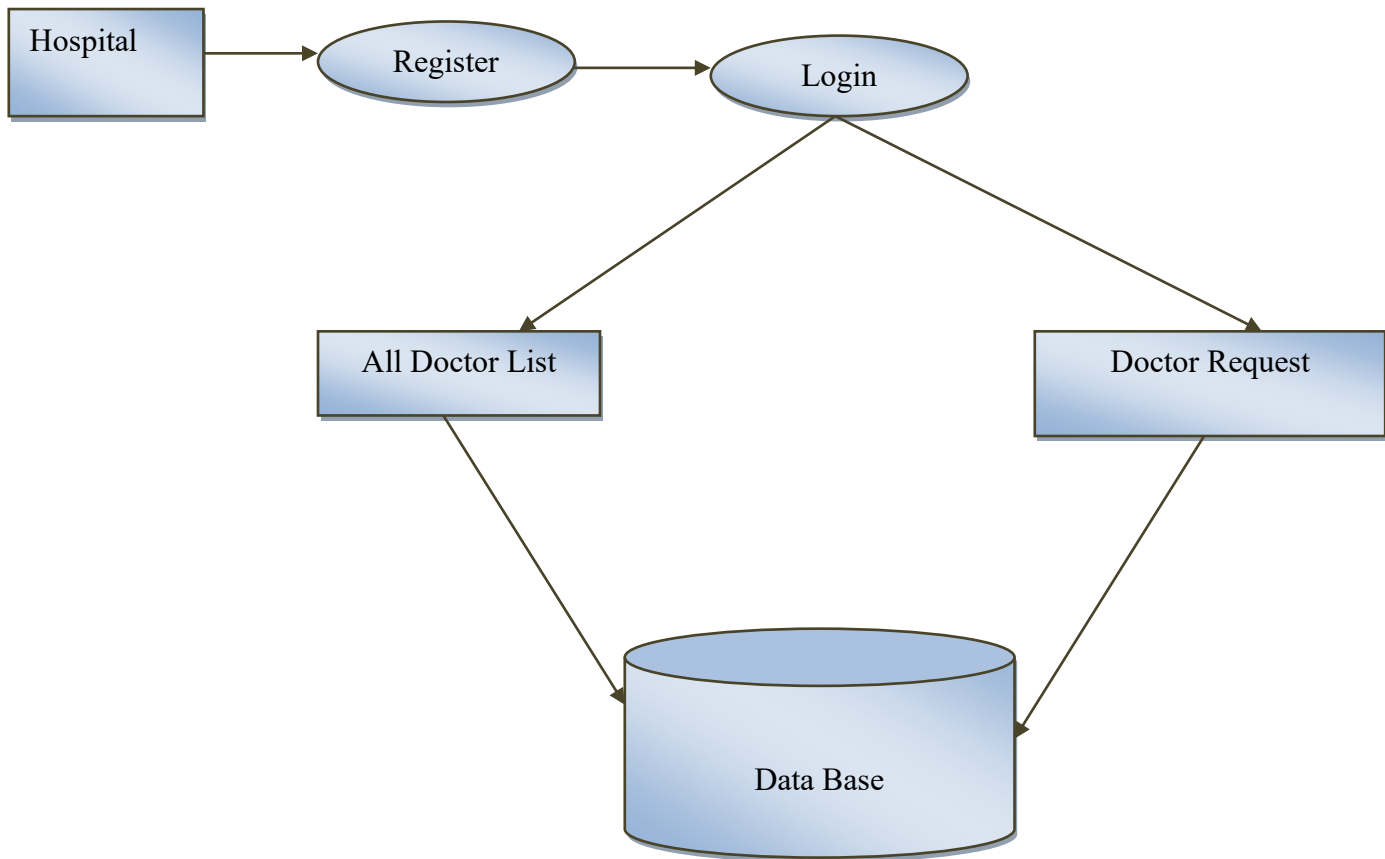
3.PATIENT

There are patient has a register with all details. Patient has a login. Patient has a takes a permission from a doctor.Doctor has a approve. Patient has a appointment status of a doctor. Patient has a upload a diseases reports. Patient has a live consulation of a doctor and patient. Patient has a history of records stores a database.



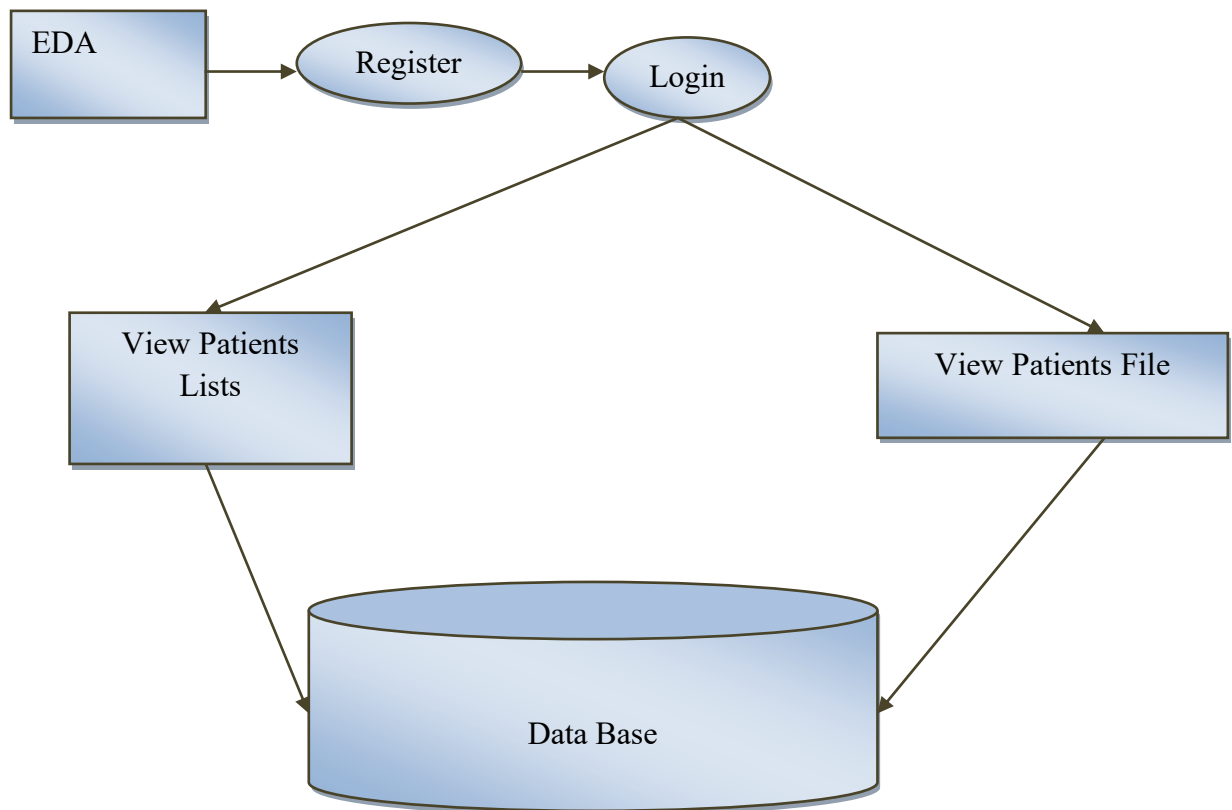
4.HOSPITAL

The Hospital has a register with a id and password. Hospital has all doctor lists. Hospital has a doctor requests.



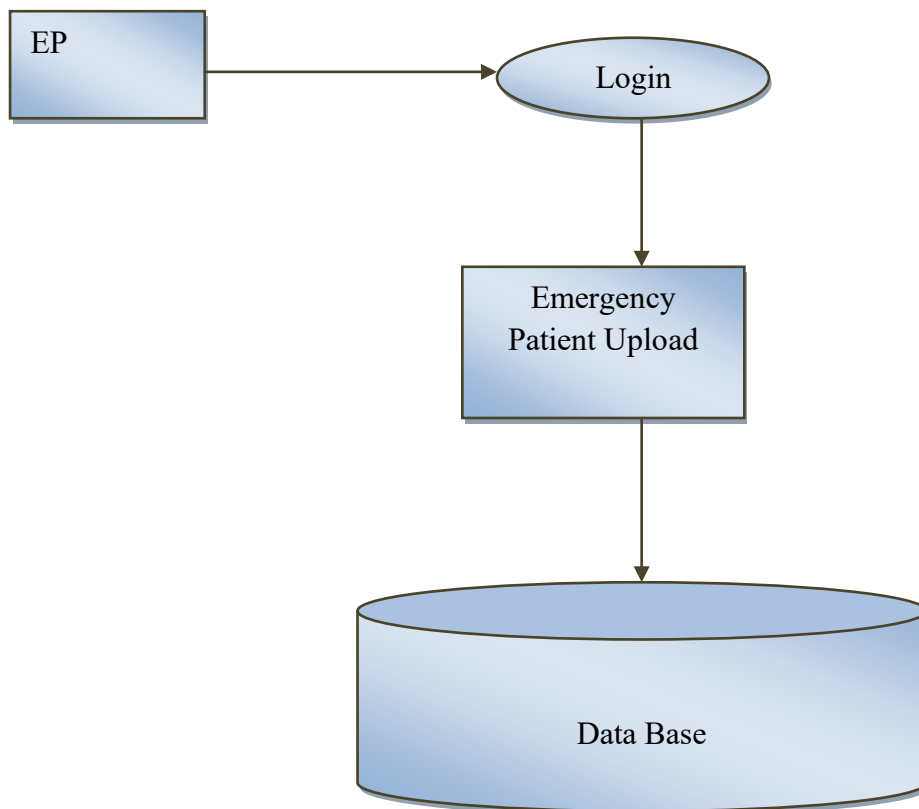
5. Emergency Doctor Admin

The emergency doctor has a login with a user id and password. Emergency doctor has a view a patient lists and Emergency doctor has a view a patient file.



6. Emergency Patient Login

The emergency patient login has a login with a user id and password. Emergency patient has a upload a emergency reports.



2.3 GIVEN INPUT EXPECTED OUTPUT:

➤ User Interface Design

Input : Enter Login name and Password

Output : If valid user name and password then directly open the home page otherwise show error message and redirect to the registration page.

➤ Doctors

Input : Doctor has a Login name and Password

Output: If valid user name and password then directly open the Doctor home page otherwise show error message and redirect to the Doctor login page.

➤ Patient

Input : Enter the Patient has a name and password

Output : If valid Patient id and password then directly open the Patient home page otherwise show error message and redirect to the Patient login page.

➤ Hospital

Input : Enter the Hospital name and password

Output: If valid Hospital id and password then directly open the Hospital home page otherwise show error message and redirect to the Hospital login page.

➤ Emergency Doctor Admin

Input : Enter the emergency doctor admin name and password

Output: If valid emergency doctor admin id and password then directly open the emergency doctor admin home page otherwise show error message and redirect to emergency doctor admin login page

2.4.1 PROPOSED ALGORITHM

1. PUBLIC KEY ENCRYPTION

In a public key (or asymmetric) encryption scheme, a sender u_1 encrypts a message m with a public key pk_{u_2} belonging to a receiver u_2 . The resulting cipher text $E(m, pk_{u_2})$ can only be decrypted by user u_2 , who owns the corresponding private key sk_{u_2} . To this end, u_2 should compute $D(E(m, pk_{u_2}), sk_{u_2})$, which returns m .

The AES-based encryption algorithm for involves the following steps:

1. **Encryption:** The data to be hidden is first encrypted using the AES algorithm with a secret key. The key must be kept secret between the sender and the receiver.
2. **Decryption:** To extract the hidden data, the receiver must first extract the encrypted data from the cover data. The extracted data is then decrypted using the same secret key that was used to encrypt the data.

This AES-based algorithm provides a high level of security for data, as the data is encrypted data. However, the algorithm is not completely secure, as it is possible for an attacker to detect the presence of data. Therefore, additional techniques such as randomization can be used to make the data more difficult to attack.

2. ONE-WAY HASH FUNCTIONS

A hash function (e.g., SHA-256) is a deterministic mathematical algorithm that receives a message m as input and outputs a fixed-size hash value $H(m)$. For a specific message m , this function always outputs the same hash value.

3. CP-ABE

In Cipher text-Policy Attribute-based Encryption (CP-ABE), data is encrypted with an access policy, a tree-based structure with attributes interrelated through logical operators, such as. Decryption occurs with the secret key of a user, composed of attributes. It is only successful if the key's attributes satisfy the access policy used for encryption. As example, let “” be an access policy. In CP-ABE, an attribute authority is responsible for generating secret keys to users with attributes that describe their profile. To this end, the authority employs a master key known only to itself

2.5 EXISTING ALGORITHM

IPFS network

In the storage phase, a collaborator generates a health record for a patient, who stores it in the IPFS network and its metadata in the blockchain in a secure fashion such that another collaborator can later access the record. After this, the designated collaborator can access the health record, in the sharing phase, if she owns the required cryptographic material for this purpose. The emergency access phase occurs when a patient needs emergency medical care and has no physical conditions (e.g., may be unconscious) to give available collaborators access permission to her health record. This phase runs in a collaborative way (between the emergency servers, collaborators and call center attendant) such that the patient can later verify who participated in the emergency session and had access to her health record. After granting access to a health record for a collaborator, the patient can revoke this access right whenever she wants by means of the access revocation mechanism in an interaction with the blockchain network. With the anonymity mechanism, a collaborator can provide a health record for research studies under patient consent.

CHAPTER 3

REQUIREMENTS ENGINEERING

3.1 GENERAL

These are the requirements for doing the project. Without using these tools and software's we can't do the project. So we have two requirements to do the project. They are

1. Hardware Requirements.
2. Software Requirements.

3.2 HARDWARE REQUIREMENTS

The hardware requirements may serve as the basis for a contract for the implementation of the system and should therefore be a complete and consistent specification of the whole system. They are used by software engineers as the starting point for the system design. It shows what the system does and not how it should be implemented.

- System : Pentium i3 Processor
- Hard Disk : 500 GB.
- Monitor : 15'' LED
- Input Devices : Keyboard, Mouse
- RAM : 2 GB

3.3 SOFTWARE REQUIREMENTS

The software requirements document is the specification of the system. It should include both a definition and a specification of requirements. It is a set of what the system should do rather than how it should do it. The software requirements provide a basis for creating the software requirements specification. It is useful in estimating cost, planning team activities, performing tasks and tracking the team's and tracking the team's progress throughout the development activity.

- Operating System : Windows 10
- Coding Language : Java
- Technologies : Core Java, JDBC, Servlets
- Web Technologies : HTML, Java Server Page, CSS, Java Script
- Tool or IDE : Eclipse
- Database : MYSQL

3.4FUNCTIONAL REQUIREMENTS

1.Doctor's Functionalities:

- Doctor registers with details (e.g., name, specialization, contact info).
- Doctor takes permission from the hospital to practice.
- Doctor grants permission to patients for consultations.
- Doctor manages patient appointments.
- Doctor can view uploaded patient reports.
- Doctor participates in live consultations with patients.
- Doctor views patient medical history.

2.Patient's Functionalities:

- Patient registers with details (e.g., name, contact info, medical history).
- Patient logs in to access the system.
- Patient takes permission from a doctor for consultations.
- Doctor approves patient consultation requests.
- Patient can book and view appointment status.
- Patient can upload disease reports.
- Patient participates in live consultations with the doctor.
- Patient's medical history is stored in a database for future reference.

3.Hospital's Functionalities:

- Hospital registers with an ID and password.
- Hospital maintains a list of all doctors.
- Hospital reviews and approves doctor requests (e.g., for practicing).

4.Emergency Doctor's Functionalities:

- Emergency doctor logs in with a user ID and password.
- Emergency doctor can view patient lists.
- Emergency doctor can view patient files for emergency consultations.

5. Emergency Patient's Functionalities:

- Emergency patient logs in with a user ID and password.
- Emergency patient can upload emergency reports for urgent care.

3.5 NON FUNCTIONAL REQUIREMENTS

1. Security:

- The system must ensure user data (doctor, patient, emergency patient) is protected using secure encryption methods (e.g., for passwords and sensitive medical data).
- Permissions and access controls should be in place to ensure that only authorized users (doctors, patients, hospital admins) can access specific information.

2. Usability:

- The user interface should be easy to navigate for doctors, patients, and hospital staff, especially for emergency situations.
- The system should have clear notifications for appointments, report uploads, and approvals.

3. Performance:

- The system should be able to handle concurrent users (doctors, patients, and emergency users) efficiently.
- The response time for appointment booking, report uploads, and consultation requests should be quick.

4. Availability:

- The system must be available 24/7, especially for emergency cases.
- Downtime should be minimized and communicated well in advance if any maintenance is required.

5. Scalability:

- The system should be scalable to handle increasing numbers of patients, doctors, and hospitals in the future.

6. Reliability:

- The system must be reliable, ensuring that all patient data, medical records, and consultation histories are stored accurately and can be accessed without issues.

7. Compliance:

- The system must comply with healthcare regulations, such as HIPAA (Health Insurance Portability and Accountability Act) in the U.S. or similar laws in other countries to ensure patient data privacy and confidentiality.

8. Backup and Recovery:

- The system should implement regular data backups and have a disaster recovery plan in place to prevent data loss.

9. Interoperability:

- The system should be able to integrate with other healthcare platforms or external systems for data sharing (such as diagnostic tools or pharmacies).

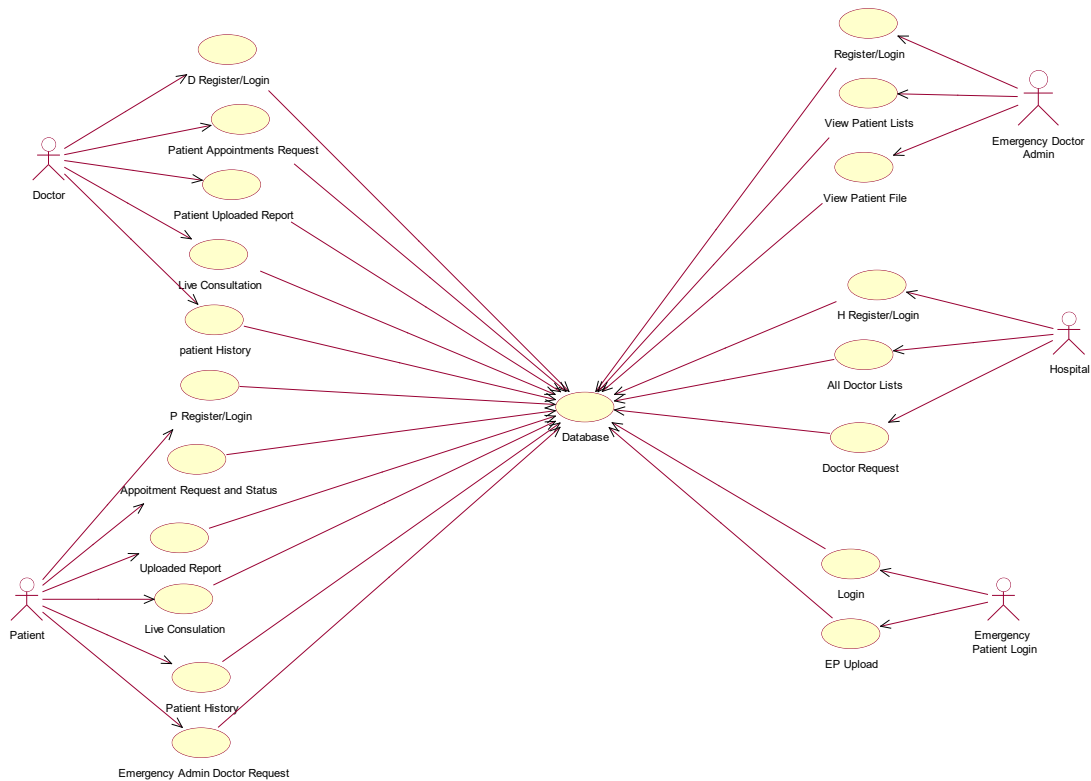
CHAPTER 4

DESIGN ENGINEERING

4.1 GENERAL

Design Engineering deals with the various UML [Unified Modeling language] diagrams for the implementation of project. Design is a meaningful engineering representation of a thing that is to be built. Software design is a process through which the requirements are translated into representation of the software. Design is the place where quality is rendered in software engineering. Design is the means to accurately translate customer requirements into finished product.

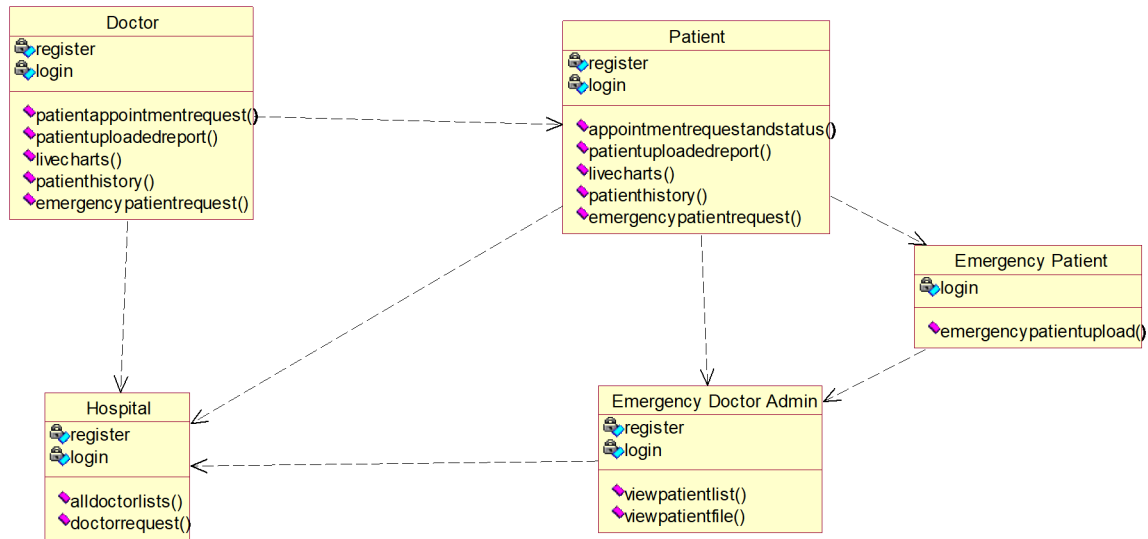
4.2.1 USE CASE DIAGRAM:



EXPLANATION:

A use case diagram is used to represent the dynamic behavior of a system. It encapsulates the system's functionality by incorporating use cases, actors, and their relationships. It models the tasks, services, and functions required by a system/subsystem of an application.

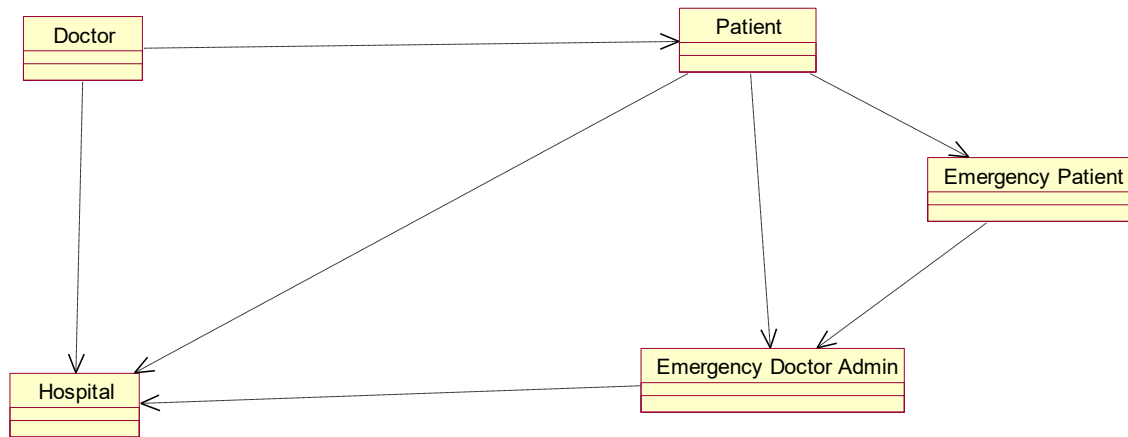
4.2.2 CLASS DIAGRAM:



EXPLANATION

The class diagram is one of the types of UML diagrams which is used to represent the static diagram by mapping the structure of the systems using classes, attributes, relations, and operations between the various objects. A class diagram has various classes; each has three-part; the first partition contains a Class name which is the name of the class or entity which is participated in the activity, the Second partition contains class attributes that show the various properties of the class, the third partition contains class operations which shows various operations performed by the class, relationships shows the relation between two classes.

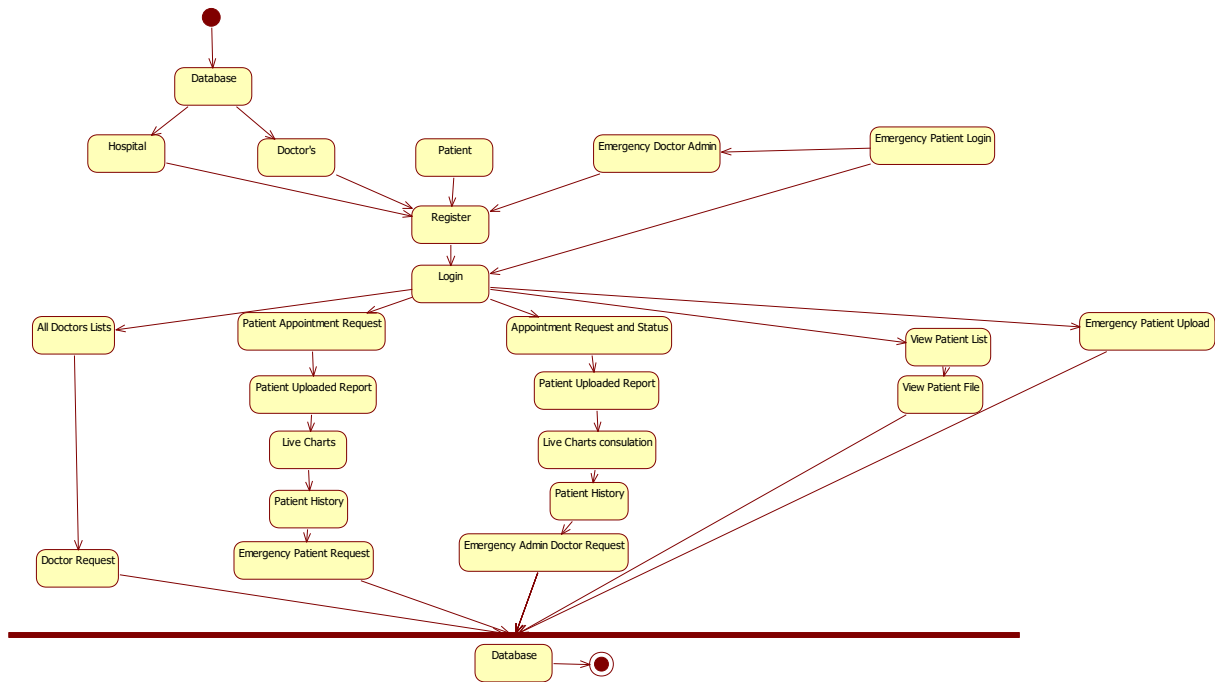
4.2.3 OBJECT DIAGRAM:



EXPLANATION:

"An object diagram is a graph of instances, including objects and data values. A static object diagram is an instance of a class diagram; it shows a snapshot of the detailed state of a system at a point in time. The use of object diagrams is fairly limited, namely to show examples of data structure..

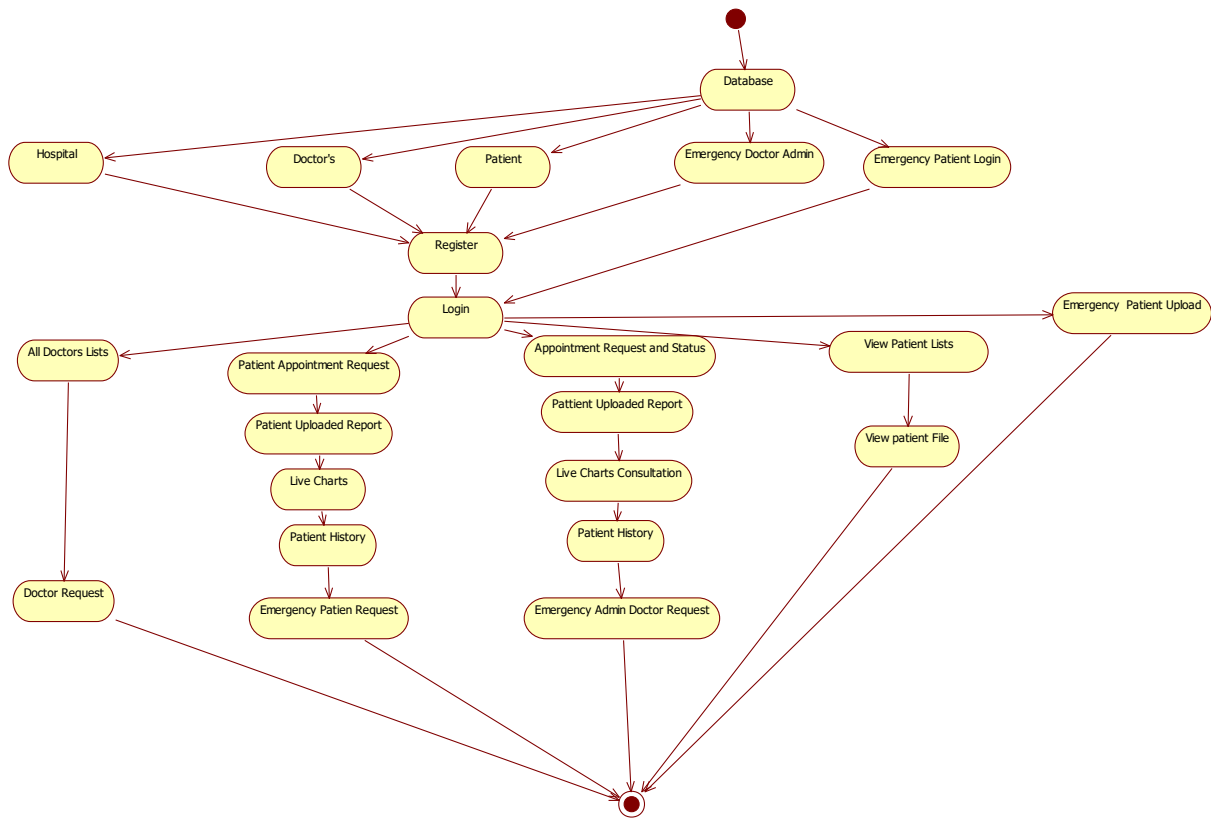
4.2.4 STATE DIAGRAM:



EXPLANATION:

A state diagram shows the behavior of classes in response to external stimuli. Specifically a state diagram describes the behavior of a single object in response to a series of events in a system. Sometimes it's also known as a Harel state chart or a state machine diagram. This UML diagram models the dynamic flow of control from state to state of a particular object within a system.

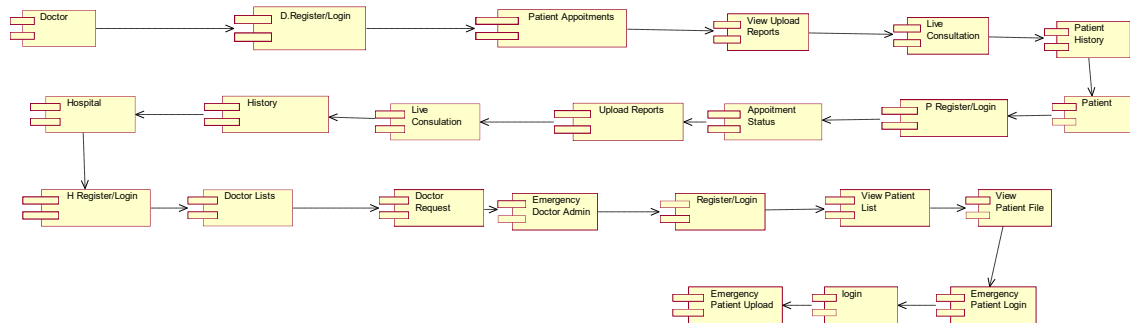
4.2.5 ACTIVITY DIAGRAM:



EXPLANATION:

It is a behavioral diagram that illustrates the flow of activities through a system. UML activity diagrams can also be used to depict a flow of events in a business process. They can be used to examine business processes in order to identify their flow and requirements. UML has specified a set of symbols and rules for drawing activity diagrams.

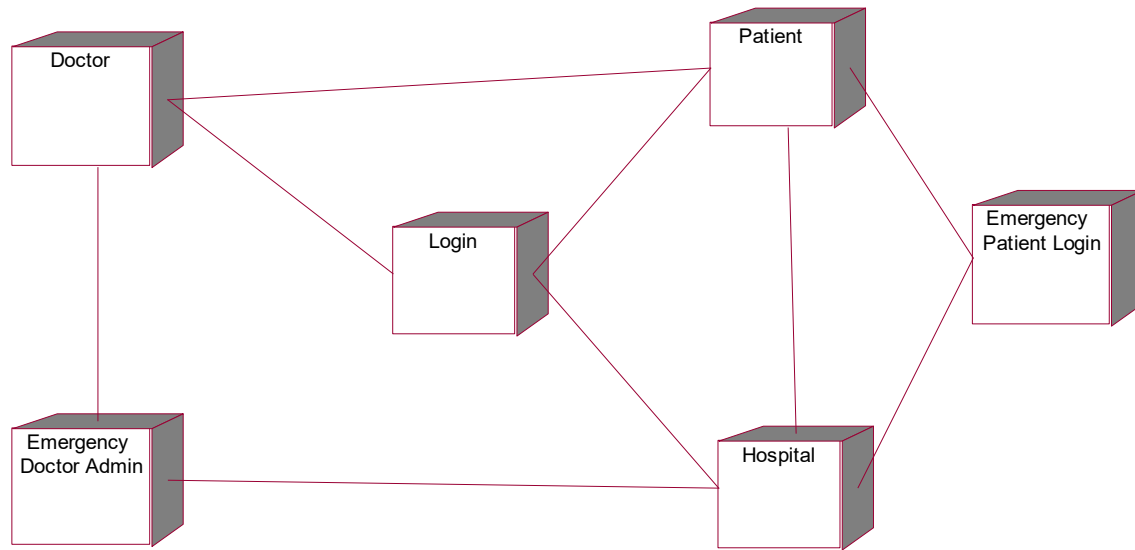
4.2.6 COMPONENT DIAGRAM



EXPLANATION

A component diagram, also known as a UML component diagram, describes the organization and wiring of the physical components in a system. Component diagrams are often drawn to help model implementation details and double-check that every aspect of the system's required functions is covered by planned development. In the first version of UML, components included in these diagrams were physical: documents, database table, files, and executables, all physical elements with a location. In the world of UML 2, these components are less physical and more conceptual stand-alone design elements such as a business process that provides or requires interfaces to interact with other constructs in the system. The physical elements described in UML 1, like files and documents, are now referred to as artifacts. A UML 2 component may contain multiple physical artifacts if they naturally belong together..

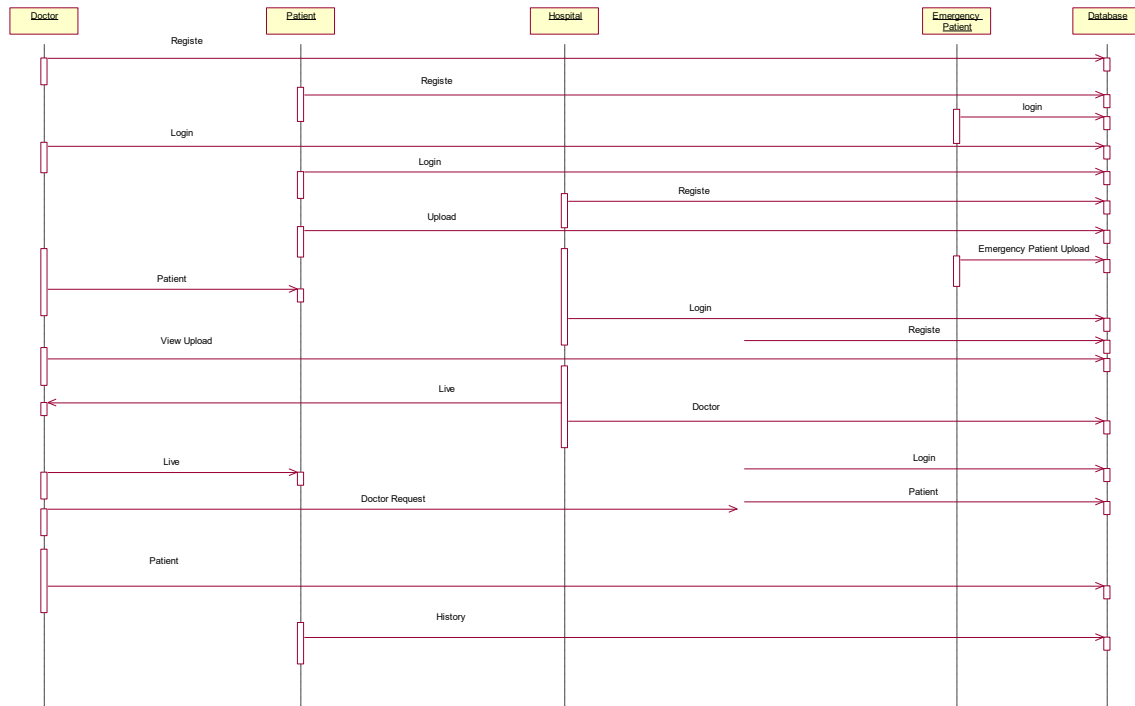
4.2.7 Deployment Diagram



EXPLANATION:

A deployment diagram is a UML diagram type that shows the execution architecture of a system, including nodes such as hardware or software execution environments, and the middleware connecting them. Deployment diagrams are typically used to visualize the physical hardware and software of a system.

4.2.8 Sequence Diagram:



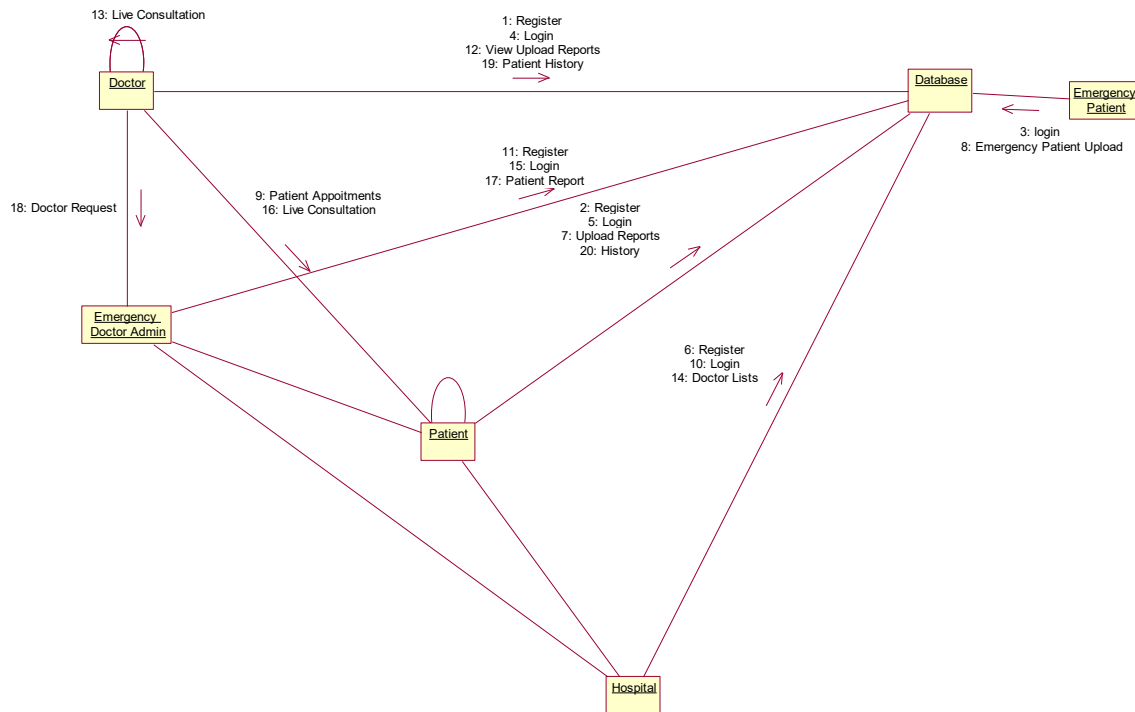
EXPLANATION:

Sequence Diagrams captures:

- the interaction that takes place in a collaboration that either realizes a use case or an operation (instance diagrams or generic diagrams)
- high-level interactions between user of the system and the system, between the system and other systems, or between subsystems (sometimes known as system sequence diagrams)

.

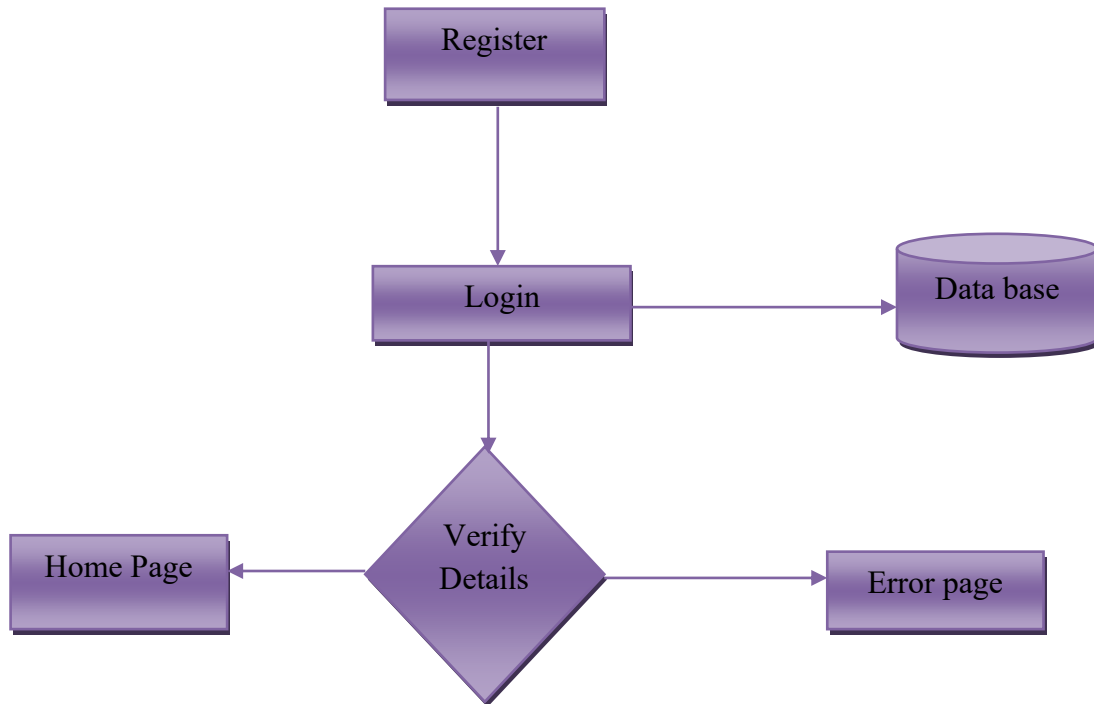
4.2.9 Collaboration Diagram:



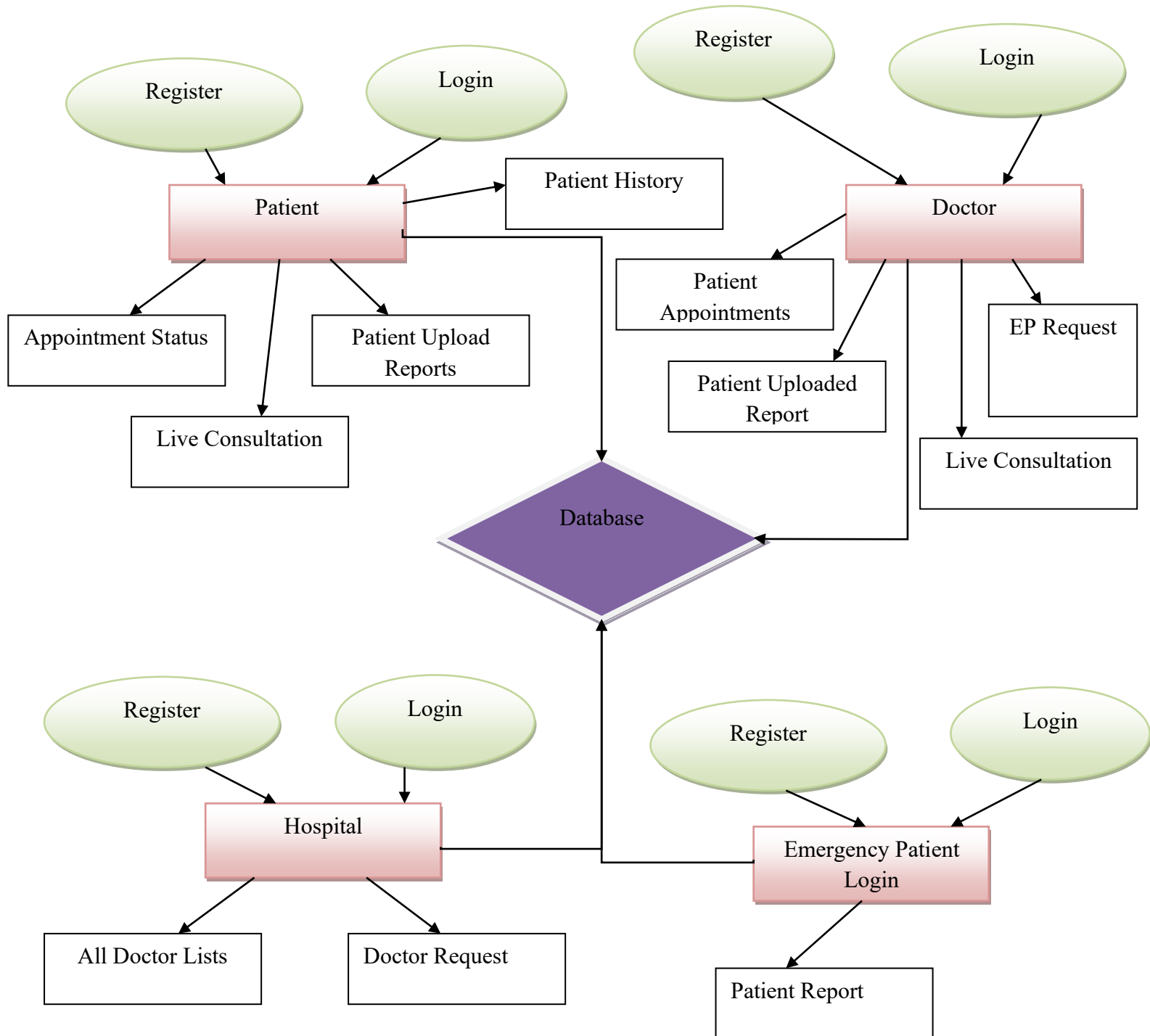
EXPLANATION:

The collaboration diagram is used to show the relationship between the objects in a system. Both the sequence and the collaboration diagrams represent the same information but differently. Instead of showing the flow of messages, it depicts the architecture of the object residing in the system as it is based on object-oriented programming. An object consists of several features. Multiple objects present in the system are connected to each other. The collaboration diagram, which is also known as a communication diagram, is used to portray the object's architecture in the system.

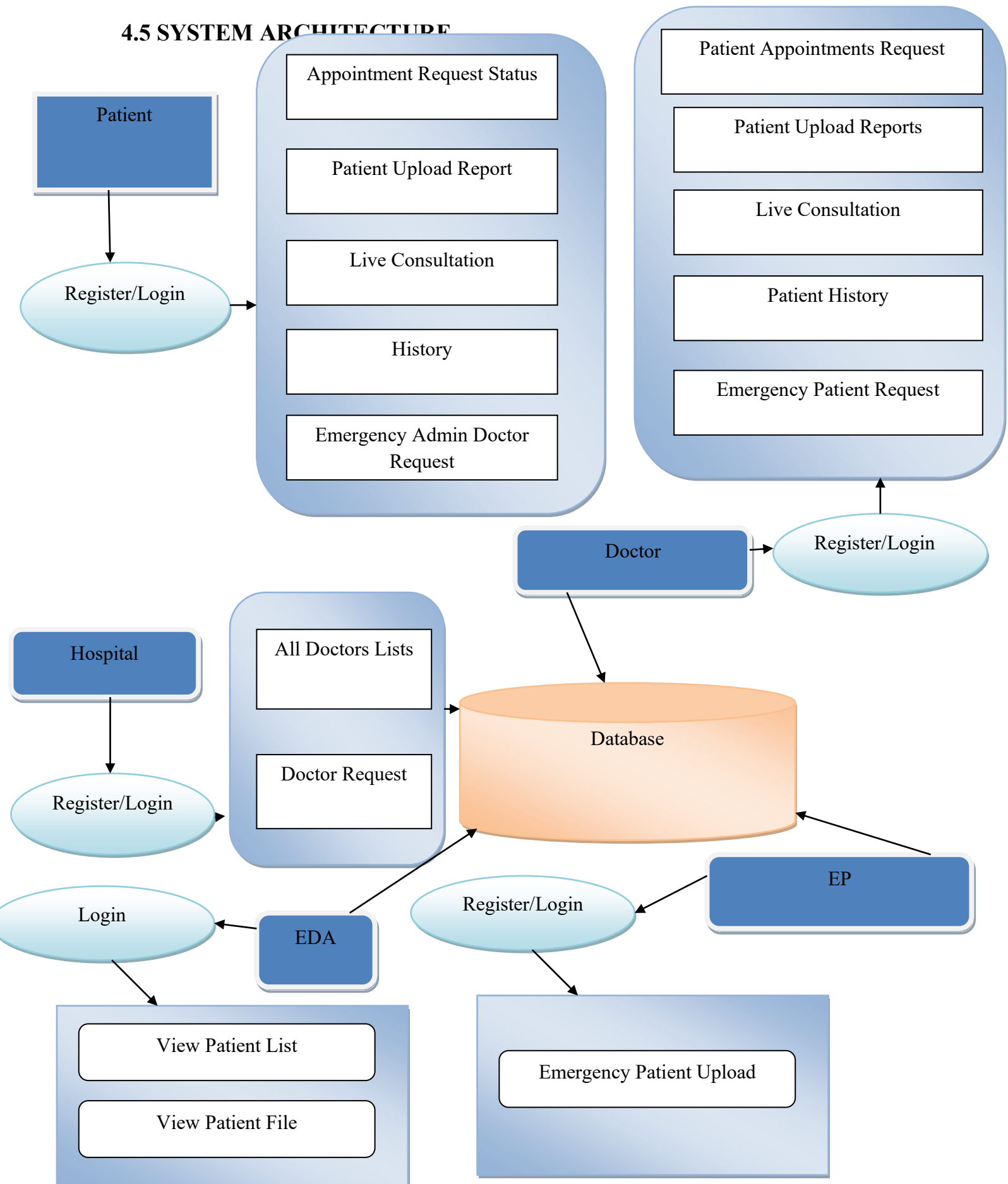
4.3 Data Flow Diagram:



4.4 ER DIAGRAM



4.5 SYSTEM ARCHITECTURE



EXPLANATION

The proposed PRMS system architecture is designed and focuses entirely on the security of patients. Doctor has a add all information. It was shared a hospital database. Hospital has a register with a all details and logins. Hospital has a all doctor lists the hospital has to approve a doctor. It will get a doctor request. Doctor has takes permission from the hospital. Doctor has a get a patient appointments. Doctor has a view upload reports. Doctor has a live consultations of a patient and doctor they will send messages and discuss a report. Doctor has a patient history. Patient has a register with all details and then login. Patient has a takes a appointment status. Patient has a upload a reports. Patient has also a live consultation. Patient has a history.

CHAPTER 5

DEVELOPMENT TOOLS

5.1 GENERAL

This chapter is about the software language and the tools used in the development of the project. The platform used here is JAVA.

5.2 FEATURES OF JAVA

5.2.1 THE JAVA FRAMEWORK

Java is a programming language originally developed by James Gosling at Sun Microsystems and released in 1995 as a core component of Sun Microsystems' Java platform. The language derives much of its syntax from C and C++ but has a simpler object model and fewer low-level facilities. Java applications are typically compiled to bytecode that can run on any Java Virtual Machine (JVM) regardless of computer architecture. Java is general-purpose, concurrent, class-based, and object-oriented, and is specifically designed to have as few implementation dependencies as possible. It is intended to let application developers "write once, run anywhere".

Java is considered by many as one of the most influential programming languages of the 20th century, and is widely used from application software to web applications. The Java framework is a new platform independent that simplifies application development on the internet. Java technology's versatility, efficiency, platform portability, and security make it the ideal technology for network computing. From laptops to datacenters, game consoles to scientific supercomputers, cell phones to the Internet, Java is everywhere!

5.2.2 OBJECTIVES OF JAVA

To see places of Java in Action in our daily life, explore java.com.

Why Software Developers Choose Java

Java has been tested, refined, extended, and proven by a dedicated community. And numbering more than 6.5 million developers, it's the largest and most active on the planet. With its

versatility, efficiency, and portability, Java has become invaluable to developers by enabling them to:

- Write software on one platform and run it on virtually any other platform
- Create programs to run within a Web browser and Web services
- Develop server-side applications for online forums, stores, polls, HTML forms processing, and more
- Combine applications or services using the Java language to create highly customized applications or services
- Write powerful and efficient applications for mobile phones, remote processors, low-cost consumer products, and practically any other device with a digital heartbeat

Some Ways Software Developers Learn Java

- Today, many colleges and universities offer courses in programming for the Java platform. In addition, developers can also enhance their Java programming skills by reading Sun's java.sun.com Web site, subscribing to Java technology-focused newsletters, using the Java Tutorial and the New to Java Programming Center, and signing up for Web, virtual, or instructor-led courses.

Object Oriented

To be an Object Oriented language, any language must follow at least the four characteristics.

1. Inheritance : It is the process of creating the new classes and using the behavior of the existing classes by extending them just to reuse the existing code and adding additional features as needed.

2. Encapsulation: It is the mechanism of combining the information and providing the abstraction.

3. Polymorphism: As the name suggests one name multiple form, Polymorphism is the way of providing the different functionality by the functions having the same name based on the signatures of the methods.

4. Dynamic binding : Sometimes we don't have the knowledge of objects about their specific types while writing our code. It is the way of providing the maximum functionality to a program about the specific type at runtime.

5.2.3JavaServer Pages - An Overview

Java Server Pages or JSP for short is Sun's solution for developing dynamic web sites. JSP provide excellent server side scripting support for creating database driven web applications. JSP enable the developers to directly insert java code into jsp file, this makes the development process very simple and its maintenance also becomes very easy.

JSP pages are efficient, it loads into the web servers memory on receiving the request very first time and the subsequent calls are served within a very short period of time.

In today's environment most web sites servers dynamic pages based on user request. Database is very convenient way to store the data of users and other things. JDBC provide excellent database connectivity in heterogeneous database environment. Using JSP and JDBC its very cceasy to develop database driven web application.

Java is known for its characteristic of "write once, run anywhere." JSP pages are platfJavaServer Pages

JavaServer Pages (JSP) technology is the Java platform technology for delivering dynamic content to web clients in a portable, secure and well-defined way. The JavaServer Pages specification extends the Java Servlet API to provide web application developers with a robust framework for creating dynamic web content on the server using HTML, and XML templates, and Java code, which is secure, fast, and independent of server platforms.

JSP has been built on top of the Servlet API and utilizes Servlet semantics. JSP has become the preferred request handler and response mechanism. Although JSP technology is going to be a powerful successor to basic Servlets, they have an evolutionary relationship and can be used in a cooperative and complementary manner.

Servlets are powerful and sometimes they are a bit cumbersome when it comes to generating complex HTML. Most servlets contain a little code that handles application logic and a lot more code that handles output formatting. This can make it difficult to separate and reuse portions of the code when a different output format is needed. For these reasons, web application developers turn towards JSP as their preferred servlet environment.

5.2.4 Evolution of Web Applications

Over the last few years, web server applications have evolved from static to dynamic applications. This evolution became necessary due to some deficiencies in earlier web site design. For example, to put more of business processes on the web, whether in business-to-consumer (B2C) or business-to-business (B2B) markets, conventional web site design technologies are not enough. The main issues, every developer faces when developing web applications, are:

1. Scalability - a successful site will have more users and as the number of users is increasing fastly, the web applications have to scale correspondingly.
2. Integration of data and business logic - the web is just another way to conduct business, and so it should be able to use the same middle-tier and data-access code.
3. Manageability - web sites just keep getting bigger and we need some viable mechanism to manage the ever-increasing content and its interaction with business systems.
4. Personalization - adding a personal touch to the web page becomes an essential factor to keep our customer coming back again. Knowing their preferences, allowing them to configure the information they view, remembering their past transactions or frequent search keywords are all important in providing feedback and interaction from what is otherwise a fairly one-sided conversation.

Apart from these general needs for a business-oriented web site, the necessity for new technologies to create robust, dynamic and compact server-side web applications has been realized. The main characteristics of today's dynamic web server applications are as follows:

1. Serve HTML and XML, and stream data to the web client
2. Separate presentation, logic and data
3. Interface to databases, other Java applications, CORBA, directory and mail services
4. Make use of application server middleware to provide transactional support.
5. Track client sessions.

5.2.5 Benefits of JSP

One of the main reasons why the JavaServer Pages technology has evolved into what it is today and it is still evolving is the overwhelming technical need to simplify application design by separating dynamic content from static template display data. Another benefit of utilizing JSP is that it allows to more cleanly separate the roles of web application/HTML designer from a software developer. The JSP technology is blessed with a number of exciting benefits, which are chronicled as follows:

1. The JSP technology is platform independent, in its dynamic web pages, its web servers, and its underlying server components. That is, JSP pages perform perfectly without any hassle on any platform, run on any web server, and web-enabled application server. The JSP pages can be accessed from any web server.
2. The JSP technology emphasizes the use of reusable components. These components can be combined or manipulated towards developing more purposeful components and page design. This definitely reduces development time apart from the At development time, JSPs are very different from Servlets, however, they are precompiled into Servlets at run time and executed by a JSP engine which is installed on a Web-enabled application server such as BEA WebLogic and IBM WebSphere.

5.3Servlets

Earlier in client- server computing, each application had its own client program and it worked as a user interface and need to be installed on each user's personal computer. Most web applications use HTML/XHTML that are mostly supported by all the browsers and web pages are displayed to the client as static documents.

A web page can merely displays static content and it also lets the user navigate through the content, but a web application provides a more interactive experience.

Any computer running Servlets or JSP needs to have a container. A container is nothing but a piece of software responsible for loading, executing and unloading the Servlets and JSP. While servlets can be used to extend the functionality of any Java- enabled server.

They are mostly used to extend web servers, and are efficient replacement for CGI scripts. CGI was one of the earliest and most prominent server side dynamic content solutions, so before going forward it is very important to know the difference between CGI and the Servlets.

5.4Java Servlets

Java Servlet is a generic server extension that means a java class can be loaded dynamically to expand the functionality of a server. Servlets are used with web servers and run inside a Java Virtual Machine (JVM) on the server so these are safe and portable.

Unlike applets they do not require support for java in the web browser. Unlike CGI, servlets don't use multiple processes to handle separate request. Servlets can be handled by separate threads within the same process. Servlets are also portable and platform independent.

A web server is the combination of computer and the program installed on it. Web server interacts with the client through a web browser. It delivers the web pages to the client and to an application by using the web browser and the HTTP protocols respectively.

The define the web server as the package of large number of programs installed on a computer connected to Internet or intranet for downloading the requested files using File Transfer Protocol, serving e-mail and building and publishing web pages. A web server works on a client server model.

CHAPTER 6

IMPLEMENTATION

6.1 GENERAL

In this we implement the coding part using eclipse. Below are the coding's that are used

6.2 PROJECT CODE

1.Hospitalbean.java

```
package com.beans;
```

```
public class HospitalBean {
```

```
private String HospitalPassword, HospitalName, HospitalID, Mobile, Country, City, Pincode, Address;
```

```
public String getHospitalPassword() {
```

```
    return HospitalPassword;
```

```
}
```

```
public void setHospitalPassword(String hospitalPassword) {
```

```
    HospitalPassword = hospitalPassword;
```

```
}
```

```
public String getHospitalName() {
```

```
    return HospitalName;
```

```
}
```

```
public void setHospitalName(String hospitalName) {
```

```
    HospitalName = hospitalName;
```

```
}
```

```
public String getHospitalID() {
```

```
    return HospitalID;
```

```
}
```

```
public void setHospitalID(String hospitalID) {  
    HospitalID = hospitalID;  
}
```

```
public String getMobile() {  
    return Mobile;  
}
```

```
public void setMobile(String mobile) {  
    Mobile = mobile;  
}
```

```
public String getCountry() {  
    return Country;  
}
```

```
public void setCountry(String country) {  
    Country = country;  
}
```

```
public String getCity() {  
    return City;  
}
```

```
public void setCity(String city) {  
    City = city;  
}
```

```
public String getPincode() {
```

```
        return Pincod;
    }
    public void setPincod(String pincod) {
        Pincod = pincod;
    }

    public String getAddress() {
        return Address;
    }

    public void setAddress(String address) {
        Address = address;
    }
}
```

2.Requestbean.java

```
package com.beans;
```

```
public class RequestBean {

    private String vm, cloud, req, date1, space;

    public String getSpace() {
        return space;
    }

    public void setSpace(String space) {
        this.space = space;
    }

    private double price;

    public String getVm() {
        return vm;
    }

    public void setVm(String vm) {
        this.vm = vm;
    }

    public String getCloud() {
        return cloud;
    }

    public void setCloud(String cloud) {
        this.cloud = cloud;
    }

    public String getReq() {
        return req;
    }

    public void setReq(String req) {
        this.req = req;
    }

}
```

```
public String getDate1() {  
    return date1;  
}  
public void setDate1(String date1) {  
    this.date1 = date1;  
}  
public double getPrice() {  
    return price;  
}  
public void setPrice(double price) {  
    this.price = price;  
}  
}
```

3.Patientlogin.java

```
package com.servlets;

import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

import com.dao.Dao;

/**
 * Servlet implementation class PatientLogin
 */
@WebServlet("/PatientLogin")
public class PatientLogin extends HttpServlet {
    private static final long serialVersionUID = 1L;

    /**
     * @see HttpServlet#HttpServlet()
     */
    public PatientLogin() {
        super();
        // TODO Auto-generated constructor stub
    }

    /**
```



```

    * @see HttpServlet#doGet(HttpServletRequest request, HttpServletResponse response)
    */

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {

        // TODO Auto-generated method stub

    }

    /**
    * @see HttpServlet#doPost(HttpServletRequest request, HttpServletResponse response)
    */

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {

        // TODO Auto-generated method stub
        PrintWriter o = response.getWriter();
        String uid = request.getParameter("uid");
        String pwd = request.getParameter("pwd");
        String sql = "select * from patients where email='"+uid+"' and
password='"+pwd+"' and status1='Approved'";
        boolean b = Dao.login(sql);
        HttpSession session = request.getSession();
        if(b == true){
            session.setAttribute("email", uid);
            sql = "select name from patients where email='"+uid+"'";
            String name = Dao.getName(sql);
            session.setAttribute("name", name);
            response.sendRedirect("PatientHome.jsp");
        }else{
            o.println("<script type='text/javascript'>");
            o.println("alert('Please enter valid Details/ Register');");
            o.println("window.location='PatientLogin.jsp';</script>");
        }
    }

```

}

}

4.Index.jsp

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Hook</title>
```

```
  <link                                                                    rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/shorthandcss@1.1.1/dist/shorthand.min.css" />
```

```
  <link                                                                    rel="stylesheet"
href="https://fonts.googleapis.com/css?family=Muli:200,300,400,500,600,700,800,900&display=swap" />
```

```
  <link rel="stylesheet" type="text/css"
```

```
    href="https://cdnjs.cloudflare.com/ajax/libs/slick-carousel/1.9.0/slick.min.css" />
```

```
  <link      rel="stylesheet"      type="text/css"      href="//cdn.jsdelivr.net/npm/slick-
carousel@1.8.1/slick/slick-theme.css" />
```

```
</head>
```

```
<body class="bg-black muli">
```

```
  <nav class="w-100pc flex flex-column md-flex-row md-px-10 py-5 bg-black">
```

```
    <div class="flex justify-between">
```

```
      <a href="#" class="flex items-center p-2 mr-4 no-underline">
```

```
        
```

```
      </a>
```

```
      <a data-toggle="toggle-nav" data-target="#nav-items" href="#"
```

```
        class="flex items-center ml-auto md-hidden indigo-lighter opacity-50 hover-opacity-
100 ease-300 p-1 m-3">
```

```
        <i data-feather="menu"></i>
```

```
      </a>
```

```
    </div>
```

```
<div id="nav-items" class="hidden flex sm-w-100pc flex-column md-flex md-flex-row md-justify-end items-center">
    <a href="index.html" class="fs-s1 mx-3 py-3 indigo no-underline hover-underline">Home</a>
    <a href="PatientLogin.jsp" class="fs-s1 mx-3 py-3 indigo no-underline hover-underline">Patient</a>
    <a href="DoctorLogin.jsp" class="fs-s1 mx-3 py-3 indigo no-underline hover-underline">Doctor</a>
    <a href="HospitalLogin.jsp" class="fs-s1 mx-3 py-3 indigo no-underline hover-underline">Hospital</a>
    <a href="RelativeLogin.jsp" class="button bg-white black fw-600 no-underline mx-5">Relative</a>
</div>
</nav>
```

```
<br><br>
<input type="submit"
      class="button-lg bg-indigo-lightest-20 indigo-lightest focus-white fw-300 fs-s3
mr-0 br-l-0" placeholder="Login"/>
  <a href="PatientRegister.jsp" class="button-lg bg-indigo-lightest-20 indigo-lightest
focus-white fw-300 fs-s3 mr-0 br-l-0">New Patient</a>
</form>
</div>
</div>
</section>

<!-- product options -->

<!-- pricing -->
<!-- testimonials -->
<!-- subscribe -->

<!-- footer -->
</body>

</html>
```


CHAPTER 7

SNAPSHOTS

CHAPTER 8

SOFTWARE TESTING

8.1 GENERAL

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

8.2 DEVELOPING METHODOLOGIES

The test process is initiated by developing a comprehensive plan to test the general functionality and special features on a variety of platform combinations. Strict quality control procedures are used.

The process verifies that the application meets the requirements specified in the system requirements document and is bug free. The following are the considerations used to develop the framework from developing the testing methodologies.

8.3 TYPES OF TESTS

Sl. No	Test scenario	User action	Expected result	Actual Result	Remarks
1.	Registration	Users registering into the system.	Register into the system.	Successfully alert registered message.	Pass
2.	Login	1. Entered correct password.	1. Log into the system. 2. Alert generated.	1. Successfully logged in. 2. Successfully generated the alert.	Pass

3.	Doctor	We can have a hospital information	Messages sending authority alert is generated.	Successfully generated the alert and messages sending	Successful
4.	Patient	Patient will have a hospital data	Patient has to actions	Successfully generated the alert to Patient server message	Successful
4.	Hospital	Store a disease,	Messages Alert is generated	Successfully generated the alert for Hospital messages	Successful
5.	Emergency Doctor Admin	EDA will have a third party of a person	EDA have a alert message is generated.	Successfully has a relative.	Successful
6.	Emergency Patient Login	EP will have a third party of a person	EP have a alert message is generated.	Successfully has a message.	Successful

8.3.1 Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program input produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

8.3.2 Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

- Valid Input : identified classes of valid input must be accepted.
- Invalid Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.
- Systems/Procedures : interfacing systems or procedures must be invoked.

8.3.3 System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

8.3.4 Performance Test

The Performance test ensures that the output be produced within the time limits, and the time taken by the system for compiling, giving response to the users and request being send to the system for to retrieve the results.

8.3.5 Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

8.3.6 Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Acceptance testing for Data Synchronization:

- The Acknowledgements will be received by the Sender Node after the Packets are received by the Destination Node
- The Route add operation is done only when there is a Route request in need
- The Status of Nodes information is done automatically in the Cache Updation process

8.2.7 Build the test plan

Any project can be divided into units that can be further performed for detailed processing. Then a testing strategy for each of this unit is carried out. Unit testing helps to identity the possible bugs in the individual component, so the component that has bugs can be identified and can be rectified from errors.

CHAPTER 9

APPLICATION

9.1 CONCLUSION

In this work, we proposed Sec-Health, a block chain-based protocol that secures health records while addressing all of their main properties, namely confidentiality, access control, integrity, access revocation, emergency access,. Sec-Health shows security advantages compared to related proposals that present highly security mechanisms. While those proposals are generally based on a server, Sec-Health affords several features, preventing one single entity from compromising the healthcare system. Furthermore, compared to solutions, our protocol addresses the challenging problem of fulfilling all the main properties of health records, whereas other solutions focus on offering mechanisms for specific only. Experimental evaluations of a Sec-Health demonstrated the practical feasibility of our protocol.

9.2 FUTURE WORK

As future work, we plan to implement the other Sec-Health mechanisms (e.g., those related to emergency access, anonymity) and evaluate them in different scenarios. Furthermore, we intend to test Sec-Health with block chain platforms of using each of them and select the best one to implement our protocol. We will also investigate types of modification we can apply to Sec-Health such that it can execute more efficiently along with the selected block chain platform. Another future work is to design a collaborative scheme to from the block chain. Finally, we plan to investigate a new version of Sec-Health in the context of quantum networks/internet..

CHAPTER 10

10.1 REFERENCES:

- [1] C. S. Kruse, A. Stein, H. Thomas, and H. Kaur, “The use of electronic health records to support population health: A systematic review of the literature,” *J. Med. Syst.*, vol. 42, no. 11, p. 214, Nov. 2018.
- [2] Y. Al-Issa, M. A. Ottom, and A. Tamrawi, “EHealth cloud security challenges: A survey,” *J. Healthcare Eng.*, vol. 2019, pp. 1–15, Sep. 2019.
- [3] HIPAA Journal. December 2021 Healthcare Data Breach Report. Accessed: Sep. 2, 2022. [Online]. Available: <https://www.hipaajournal.com/december-2021-healthcare-data-breach-report/>
- [4] I. M. Lopes, T. Guarda, and P. Oliveira, “General data protection regulation in health clinics,” *J. Med. Syst.*, vol. 44, no. 2, p. 53, Feb. 2020.
- [5] S. Mhatre and A. V. Nimkar, “Secure cloud-based federation for EHR using multi-authority ABE,” *Progress in Advanced Computing and Intelligent Engineering (Advances in Intelligent Systems and Computing)*, vol. 714. Singapore: Springer, 2019. [Online]. Available
- [6] R. Ganiga, R. Pai, M. Pai, and R. Sinha, “Security framework for cloud based electronic health record (EHR) system,” *Int. J. Electr. Comput. Eng.*, vol. 10, pp. 455–466, Feb. 2020.
- [7] V. Patel, “A framework for secure and decentralized sharing of medical imaging data via blockchain consensus,” *Health Informat. J.*, vol. 25, no. 4, pp. 1398–1411, Dec. 2019.
- [8] B. Shen, J. Guo, and Y. Yang, “MedChain: Efficient healthcare data sharing via blockchain,” *Appl. Sci.*, vol. 9, no. 6, p. 1207, Mar. 2019.
- [9] S. Nakamoto. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. Accessed: Sep. 7, 2022. [Online]. Available.
- [10] J. Benet, “IPFS—Content addressed, versioned, P2P file system,” 2014, arXiv:1407.3561.
- [11] J. Bethencourt, A. Sahai, and B. Waters, “Ciphertext-policy attribute-based encryption,” in *Proc. IEEE Symp. Secur. Privacy*, Oakland, CA, USA, Dec. 2007, pp. 321–334.
- [12] L. da Costa, B. Pinheiro, R. Araujo, and A. Abelem, “A decentralized protocol for securely storing and sharing health records,” in *Proc. IEEE Int. Conf. E-Health Netw., Appl. Services (HealthCom)*, Bogotá, Colombia, Oct. 2019, pp. 1–6.

- [13] W. B. Lee and C. D. Lee, “A cryptographic key management solution for HIPAA privacy/security regulations,” *IEEE Trans. Inf. Technol. Biomed.*, vol. 12, no. 1, pp. 34–41, Jan. 2008.
- [14] LGPD. (2018). Lei no 13.709, de 14 de Agosto de 2018 (in Portuguese). Accessed: Sep. 7, 2022. [Online]. Available: http://www.planalto.gov.br/ccivil_03/_Ato2015-2018/2018/Lei/L13709.htm
- [15] K. Yang, X. Jia, and K. Ren, “Attribute-based fine-grained access control with efficient revocation in cloud storage systems,” in *Proc. 8th ACM Symp. Inf., Comput. Commun. Secur. (ASIA CCS)*, K. Chen, Q. Xie, W. Qiu, N. Li, W.-G. Tzeng, Eds. Hangzhou, China, May 2013, pp. 523–528.
- [16] Y. Yang, X. Zheng, W. Guo, X. Liu, and V. Chang, “Privacy-preserving smart IoT-based healthcare big data storage and self-adaptive access control system,” *Inf. Sci.*, vol. 479, pp. 567–592, Apr. 2019.
- [17] R. L. Rivest, A. Shamir, and L. M. Adleman, “A method for obtaining digital signatures and public-key cryptosystems (reprint),” *Commun. ACM*, vol. 26, no. 1, pp. 96–99, 1983.
- [18] FIPS. (2002). Secure Hash Standard. [Online]. Available: <https://csrc.nist.gov/csrc/media/publications/fips/180/2/archive/2002-08-01/documents/fips180-2.pdf>
- [19] M. Jakobsson and A. Juels, “Mix and match: Secure function evaluation via ciphertexts,” in *Advances in Cryptology—ASIACRYPT (Lecture Notes in Computer Science)*, vol. 1976, T. Okamoto, Ed. Kyoto, Japan: Springer, Dec. 2000, pp. 162–177.
- [20] D. Johnson, A. Menezes, and S. Vanstone, “The elliptic curve digital signature algorithm (ECDSA),” *Int. J. Inf. Secur.*, vol. 1, no. 1, pp. 36–63, Aug. 2001.
- [21] X. Wang, X. Zha, W. Ni, R. P. Liu, Y. J. Guo, X. Niu, and K. Zheng, “Survey on blockchain for Internet of Things,” *Comput. Commun.*, vol. 136, pp. 10–29, Feb. 2019.
- [22] G. W. Peters and E. Panayi, “Understanding modern banking ledgers through blockchain technologies: Future of transaction processing and smart contracts on the Internet of Money,” 2015, arXiv:1511.05740.
- [23] Hyperledger. Hyperledger Fabric. Accessed: Sep. 7, 2022. [Online]. Available.

- [24] M. Masud, G. S. Gaba, K. Choudhary, R. Alroobaea, and M. S. Hossain, “A robust and lightweight secure access scheme for cloud based Ehealthcare services,” *Peer-Peer Netw. Appl.*, vol. 14, no. 5, pp. 3043–3057, Sep. 2021.
- [25] Y. Liu, Y. Zhang, J. Ling, and Z. Liu, “Secure and fine-grained access control on E-healthcare records in mobile cloud computing,” *Future Gener. Comput. Syst.*, vol. 78, pp. 1020–1026, Jan. 2018.
- [26] M. Kumar and S. Chand, “A secure and efficient cloud-centric Internet-ofMedical-things-enabled smart healthcare system with public verifiability,” *IEEE Internet Things J.*, vol. 7, no. 10, pp. 10650–10659, Oct. 2020.
- [27] H. Qiu, M. Qiu, M. Liu, and G. Memmi, “Secure health data sharing for medical cyber-physical systems for the healthcare 4.0,” *IEEE*